

How a complaint becomes a triage level

Every stage from raw Roman Urdu to a percentage on screen — which model runs where, which parameters are actually sent, and exactly what the confidence number is computed from.

READ THIS FIRST

There are two pipelines, and they share almost nothing

The single most confusable thing in this project: two separate model bundles, two different sentence encoders, both emitting **384 dimensions**. A vector from one will load into the other, predict, and be wrong, with nothing to signal it.

BUNDLE A · THE GUI'S MAIN TAB

Hybrid, 10,000 rows

The deployed system. Reads vitals, categoricals and complaint text together. This is what "Triage this patient" runs.

dir	triage_model_embedding/
method	D) dictionary + embeddings
encoder	MiniLM-L12-v2
data	cardiac_..._v3.csv (synthetic)
output	Triage_Level 1-4

BUNDLE B · SRC/ AND RUN_INFERENCE.PY

Embedding-only, 185 rows

The professor-baseline reproduction. Text is the *only* input — no vitals reach it. Also predicts department.

dir	models_src/
estimator	RandomForest x2
encoder	e5-small, "passage: "
data	dataset.csv, 185 rows
output	Level 1-4 + 11 departments

Each bundle reads its encoder name from its own `manifest.json` rather than a constant, so the two spaces cannot be swapped by accident. `embed_for_classifier()` is kept separate from `embed_text()` for the same reason.

Seven stages, in the order they execute

Stages 2–3 only run in English mode. In Roman Urdu mode the typed text goes straight to stage 4 and Ollama is never contacted.

1 triage_gui.py · form submit

Collect the patient record

Six vitals (age, heart rate, systolic BP, diastolic BP, temperature, SpO₂), four categoricals (gender, mode of arrival, AVPU, ECG status), and the free-text complaint. Unreadable or missing vitals fall back to the **training mean**, and every substitution is recorded in Notes — a triage level computed from a vital the patient never had must not look identical to a real one.

2 src/offline_pipeline.py · select_translation_model()

Pick a local model

Queries `GET /api/tags` and matches against a preference chain — `llama3.2` first, then med-translator, qwen2.5, mistral, gemma, phi3. Calling a tag that isn't pulled returns a 404 that reads like a server fault, so resolution happens before the request, and the CLI reports the *resolved* tag rather than the requested one.

3 src/offline_pipeline.py · translate_roman_urdu()

Translate on localhost

One `POST /api/chat`. The message array is doing load-bearing work here, not just carrying the text — see the next section.

4 triage_pipeline.py · build_text_features()

Normalize, then encode

Clean → fuzzy dictionary mapping → stop-word removal, then encode. A bundle trained with `skip_normalization` must be *served* that way: feeding the English model text pushed back through the Roman Urdu dictionary ("chest pain" → "sēna dārd") was a train/serve skew that cost **38 points of accuracy**, 52.5% → 90.5%.

5 triage_pipeline.py · predict_one()

Assemble one feature vector

Four blocks concatenated in a fixed order that must match training exactly.

6 predict_proba → argmax

Predict, then cap

The class probability is the confidence. If the complaint carries no usable text signal, it is capped — a model that scores a blank complaint at 94% is worse than useless.

7 src/offline_pipeline.py · run()

Compare the two language paths

Cosine between the Roman Urdu and English vectors decides which prediction is accepted. Read the limitation section before trusting this stage.

STAGE 3, IN DETAIL

What is actually sent to Ollama

llama3.2 refused outright on cardiac complaints — *"I can't provide medical advice. If you are experiencing chest pain, please seek medical attention immediately."* System-prompt instructions did not stop it. Two changes to the **request shape** did.

```
POST http://localhost:11434/api/chat

model      : llama3.2:latest          resolved, never the requested tag
stream     : false
timeout    : 120 s
options    : { temperature: 0.0 }    same input → same English, every run

messages  :
  system    SYSTEM_PROMPT (5,723 chars – anatomical dictionary,
                           phonetic variants, no clinical inference)
  user      <record>gala kharab hai aur khansi</record>    1 few-shot
  assistant Sore throat and cough                        | as real
  user      <record>taang mein sujan hai</record>          | turns
  assistant Leg swelling                                  1
  user      <record>{the patient's complaint}</record>
```

- **The `<record>` wrapper.** Handed a bare complaint, the model reads a person in the room asking for help. Tagged as stored text, it converts without hesitating.
- **Few-shot as conversation turns, not prose.** Two completed conversions establish by demonstration that records get converted. Instructions describing that rule did not work; turns showing it did.
- **Those two examples use throat and leg** — anatomy that appears nowhere in the test battery. Reusing a test sentence would let the model echo the answer back and score a hit that proves only that it can copy.

Then a three-layer net: one retry with an explicit non-refusal instruction, and `sanitize_translation()` filtering per sentence. A good translation with "Please consult a doctor" bolted on keeps the translation; a whole-response refusal returns `None`, which routes into the fallback every caller already has. Passing it through would embed *"I am not a medical professional"* as though it were the patient's symptoms.

Refusal patterns are anchored to a refusal-specific object — `cannot_translate`, never bare `cannot`. A real complaint legitimately translates to "I cannot walk", and a loose pattern would silently delete the most urgent line in the record. Result: **8/8 complaints translate, zero refusals.**

STAGE 5, IN DETAIL

The feature vector

Bundle A concatenates four blocks. Block order is fixed by `REPRESENTATION_BLOCKS` and must match the `np.hstack` order used at training time.

NUMERIC 6 vitals StandardScaler	CATEGORICAL 4 fields one-hot	BAG OF WORDS word + char n-grams × attention weights	EMBEDDING MiniLM-L12-v2, 384-d L2-normalised
--	---	---	---

Two details that are easy to get wrong. Categoricals are **one-hot, not ordinal** — encoding ECG_Status as an ordinal integer into a linear model scored 49.2% against 84.7% one-hot, because it asserts an ordering between rhythms that does not exist. And each text block gets its own `block_scaler` before concatenation, so a 384-dimension embedding block cannot drown out six vitals purely on width.

Bundle B skips all of this. Its vector *is* the 384-dimension e5-small embedding, with the `"passage: "` prefix the model was trained with. No vitals reach it — which is why its department output can disagree with the main tab.

STAGE 6, IN DETAIL

Where the percentage comes from

The confidence on screen is the winning class's probability, and nothing more sophisticated than that.

```
proba      = model.predict_proba(X)[0]      # e.g. [0.08, 0.56, 0.27, 0.09]
level      = int(np.argmax(proba))          # 1 → Level 2, URGENT
confidence = float(proba[level])             # 0.558 → "55.8%"

if not has_text_signal(complaint):
    confidence = min(confidence, MAX_CONFIDENCE_WITHOUT_TEXT)  # 0.50
```

For a RandomForest, that probability is the share of trees voting for the winning class — an ensemble agreement measure. It is **not calibrated**: 55.8% does not mean this patient is Level 2 in 55.8% of comparable cases. Treat it as a relative signal between predictions, never as a probability you could act on numerically.

The cap is the one place the number is overridden. A complaint of `"..."` or `"n/a"` still produces a full feature vector — the vitals alone can push the forest to 94% — and the resulting screen is indistinguishable from a real reading. Capping at 0.50 plus an explicit warning makes the absence visible. Bundle B has no equivalent cap; instead the CLI prints a warning under 40%, because 11 departments across 185 rows cannot support a confident answer.

LEVEL	LABEL	TARGET
1	Emergency	immediate
2	Urgent	within 15 min
3	Standard	within 60 min
4	Non-urgent	can wait / redirect

STAGE 7, IN DETAIL

The embedding gate — and why it does not work

Both vectors are L2-normalised, so the dot product *is* the cosine. If it clears the threshold the English prediction is accepted; otherwise the pipeline falls back to the Roman Urdu one, which is the language the classifiers were actually fitted on.

```
sim = float(np.dot(vec_roman_urdu, vec_english))

DEFAULT_THRESHOLD = 0.75
FALLBACK_THRESHOLD = DEFAULT_THRESHOLD    # alias, not a second literal

accepted = "english" if sim >= DEFAULT_THRESHOLD else "roman_urdu"
```

The threshold was 0.90, which is the right bar for "are these the same sentence" and the wrong one for a cross-language pair: the same complaint in both languages sits near 0.80 on e5-small even when the translation is flawless. At 0.90 the gate rejected correct translations. Lowering it to 0.75 fixed that — and exposed something worse.

Measured, not theoretical: this gate currently blocks nothing

Against `seena mein shadeed dard aur pasina` (severe chest pain with sweating):

CANDIDATE ENGLISH	COSINE	GATE
"Severe chest pain and sweating."	0.8054	PASS
"I have a headache and dizziness."	0.7908	PASS
"My leg is broken after a fall."	0.7922	PASS

Correct and catastrophically wrong sit **0.013 apart**, and the badly drifted candidate outscores the mildly drifted one. No threshold value separates them — 0.90 rejects everything, 0.75 accepts everything. The encoder's cross-language geometry does not encode translation fidelity at this resolution.

A viewer reading "PASS @ 0.75" on a demo screen will believe a safety check ran. The fix is not a number: a deterministic anatomical check — pull `seena` / `pait` / `sar` from the source with the dictionary already in the prompt, and assert the matching English term appears in the output — catches chest → head with certainty. Cosine will not.

STANDING CAVEATS

What this system is not

- **The training data is synthetic.** `cardiac_multilingual_10000_v3.csv` was produced by `generate_cardiac_dataset.py`, seed 20260816. These are not real patient records and must not be described as such in any publication or evaluation.

- **Bundle B's department head is weak.** 11 departments over 185 rows. It called a correctly-translated stomach complaint *Infectious_Disease* at 14% confidence.
- **The English bundle was trained on gpt-4o-mini translations.** Serving it locally through Ollama does not change what it was fitted on, which is why the banner still says so.
- **Known translation defect:** kandhay (shoulder) renders as "arm".
- **Research prototype, not a medical device.**

Every figure on this page was measured against the running code, not estimated. Accuracy and under-triage figures come from stratified 5-fold cross-validation; the 8/8 translation and guardrail results are single-run, temperature 0.